

Laboratório de Concorrência · Aula 10

Rayan de Andrade e Andrade · RA 24211898-2 · Turma Turma C

Gerado em 09/10/2026, 20:09:47 · tempo registrado no cronômetro: 51:14

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A diferença entre os dois é essencialmente que: Enquanto em uma delas(competição) os recursos "brigam" entre si para conseguirem o acesso do que precisam (podendo ocorrer um deadlock), a outra (Cooperação) se conversam para que o mesmo recurso não seja disputado, e sim compartilhado.

Exemplo de competição: Duas pessoas tentando pegar agua no mesmo bebedouro ao mesmo tempo

Exemplo de cooperação: Um descarregador de caminhão depende que outro pegue o que precisa para que ele possa pegar a caixa seguinte, assim esvaziando o caminhão mais rápido.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Um valor bem menor que 2000000, mas não sei dizer qual.

1ª execução (n = 1.000.000)

1039116

2ª execução (n = 1.000.000)

1082042

3ª execução (n = 1.000.000)

1030004

Saída do programa

Quando mudamos para 1000:

```
esperado: 2000
obtido: 2000
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valores não batem com o esperado. As três execuções obtiveram resultados diferentes devido ao jeito que a função foi feita.

Pelo processo de leitura, somatória e gravação do valor ser algo a ser competitivo, e não algo compartilhado de forma cooperativa, diversas vezes durante o programa ele não consegue finalizar sua ação.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

O programa 1 inicia, lê. O programa 2 inicia, lê. O programa um soma e possivelmente grava ao mesmo tempo. O programa 2 faz o mesmo. Pode ser que, antes do programa 1 terminar, o programa 2 termina, invalidando o valor somado dentro do programa um, e sobrescrevendo ele. Assim, ao invés de ser programa1, programa2, programa1,

programa2... se torna programa1, programa2, programa2, programa1, programa1, programa2, aonde as ordens podem não "obedecer" por motivos de competição.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n=1000$ o valor esperado foi correto, pois como o runtime dele é menor, os recursos conseguem ser melhor organizados. O Erro é perigoso pela possível perda de dados.

Um usuário tenta fazer uma compra, e outro tenta ao mesmo tempo. apenas um dos usuários tem o valor debitado do cartão.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AABABB → 6

AAABBB → 8

ABBABA → 4

AABBBA → 4

BBABAA → 4

BBBAAA → 7

BBAAAB → 6

BBAABA → 4

Saída do programa

a

```
AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBBA
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens diferentes, 9 delas como 4, 9 como 6, 1 como 7 e apenas 1 como 8

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

BBAABA.

A operação B fez o que precisava, mas não gravou antes de A começar. Quando B grava, ele logo é sobrescrito pelo resultado da operação B

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Monitor (synchronized)

Saída do programa

a

```
esperado:          2000000
sem sincronizacao: 1323428 em 5 ms
synchronized:      2000000 em 36 ms
AtomicInteger:     2000000 em 39 ms
```

```
esperado:          2000000
sem sincronizacao: 1018831 em 2 ms
synchronized:      2000000 em 91 ms
AtomicInteger:     2000000 em 78 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As que deram o valor correto foram:

Monitor e operação atômica.

A versão correta é mais lenta pois precisa dividir os recursos entre as duas execuções. A versão sem sincronização é mais rápida pois ela apenas faz, sem pensar se poderia sobrescrever ou não.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Teoricamente sim, pois a ação seria feito com resultado feliz, mas o certo seria sincronizado, para que os valores fossem atualizados "ao mesmo tempo".

Por fazer cada operação de cada vez, o Atomic seria suficiente.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não. Pois as duas não conseguirão ter os dois ao mesmo tempo.

Saídas (antes e depois da correção)

a

```
IMPASSE: 2 threads presas
```

```
A terminou  
B terminou  
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas Threads ficam presas pois uma tem o recurso X, e quer pegar o recurso Y. Enquanto a outra tem o Recurso Y e quer pegar o recurso X. As duas querem eles ao mesmo tempo, então não deixam o recurso "solto" para que a outra possa o pegar.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos na mesma ordem elimina o impasse pois dessa maneira elas conseguem "soltar" o recurso para que a outra consiga pegar. Então quem começou primeiro consegue terminar para que a segunda consiga dar seguimento e então, futuramente, terminar.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Espero que demore apenas 200ms. Pois cada trabalhador faria a tarefa em 100ms, e então pegaria a segunda. com um trabalhador demoraria 600ms e com 6 demoraria apenas 100ms.

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

a

```
trabalhador 2 fez a tarefa 3  
trabalhador 3 fez a tarefa 1  
trabalhador 1 fez a tarefa 2  
trabalhador 2 fez a tarefa 4  
trabalhador 1 fez a tarefa 6  
trabalhador 3 fez a tarefa 5  
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

```
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

```
trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
trabalhador 5 fez a tarefa 6
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Os três tempos são diferentes pois há números diferentes de trabalhadores fazendo tarefas Simultaneamente. Esse programa não precisa de synchronized nem de trava pois os channels fazem a comunicação necessária para que não ocorra conflito entre os trabalhadores pegando a mesma tarefa para fazer a operação.

Saídas (antes e depois da troca)

a

```
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2
```

The screenshot shows a Go IDE with a file named 'main.go'. The code defines a channel 'ch' with a buffer of 1 and a goroutine that sends a message to it. The main function sends a message to the channel and then prints the received message. The output on the right shows the number '1', indicating that the message was received successfully.

```
1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int, 1) // canal SEM espaço (sem buffer)
9     ch <- 1                 // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava pois o canal não tem espaço para seu buffer, não conseguindo receber nenhuma mensagem de resposta após enviar. O canal com espaço para mensagem resolve pois daí o channel consegue receber a mensagem de volta e então ser fechado.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Ocorrerá um impasse.

Saídas (o erro e o resultado com move)

a

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is
owned by the current function
--> HelloWorld.rs:7:27
  |
7 |     let t = thread::spawn(|| {
  |                               ^^ may outlive borrowed value `contador`
8 |         contador += 1;           // a thread altera a variável de main
  |         ----- `contador` is borrowed here
  |
note: function requires argument type to outlive ``static`
--> HelloWorld.rs:7:13
  |
7 |     let t = thread::spawn(|| {
  |     _____^
8 | |         contador += 1;           // a thread altera a variável de main
9 | |     });
  | |     ^
help: to force the closure to take ownership of `contador` (and any other referenced
variables), use the `move` keyword
  |
7 |     let t = thread::spawn(move || {
  |                             +++++

error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
--> HelloWorld.rs:11:20
  |
7 |     let t = thread::spawn(|| {
  |     -                                     -- mutable borrow occurs here
  |     _____|
  | |
8 | |         contador += 1;           // a thread altera a variável de main
  | |         ----- first borrow occurs due to use of `contador` in closure
9 | |     });
  | |     - argument requires that `contador` is borrowed for ``static`
10 |     t.join().unwrap();
11 |     println!("{}", contador);
  |                   ^^^^^^^^^ immutable borrow occurs here
  |
note: requirement that the value outlives ``static` introduced here
-->
```

```

warning: value captured by `contador` is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1;           // a thread altera a variável de main
|         ^^^^^^^
= help: did you mean to capture by reference instead?
= note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default

warning: value assigned to `contador` is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1;           // a thread altera a variável de main
|         ^^^^^^^^^^^^^
= help: maybe it is overwritten before being read?

warning: 2 warnings emitted

0

```

Saída do programa

a

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Pois o programa, segundo o erro apresentado, faz com que contador +=1 nunca seja lido.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Rust o Erro foi descoberto ainda na compilação do código, enquanto na etapa de desenvolvimento.

A vantagem é que o programa pode evitar esse problema, não tendo o perigo de que alguma hora em um ambiente de produção o problema venha a tona e possa danificar a reputação do programa. O custo são horas de sono e Retrabalho

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A,B,D,C

Saída do programa

a

```
▼ Console
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Saíram na ordem: A, D, C, B. O B saiu depois pois no console.log, ele é executado imediatamente. C então é executado após o programa, e B apenas quando o programa toma timeOut

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Pois a função Ocupado fez com que o programa demorasse 300ms para finalizar. Significa que se o programa tiver gargalos desse tipo, pode ser que ele demore muito para ser responsivo para o usuário

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O programador, com sincronização

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Impasse. Eu evitaria ele fazendo com que o ingresso estivesse vinculado a sessão da pessoa, e essa sessão seria um channel. Não haveria como as duas pessoas clicarem em comprar ao mesmo tempo se elas estivessem em ingressos diferentes. No caso delas clicassem no ingresso ao mesmo tempo, seria feito pela conexão de rede delas, e quem obteve menos latência em ms.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

A última, pelo fato de haver apenas uma Thread em JS.